

SQL 5 : Fonctions et procédures

Guillaume Guex guillaume.guex@unil.ch

Les fonctions

Une **fonction** est une suite d'instructions SQL qui donne en retour une valeur. Il existe plusieurs **fonctions intégrées** à SQL (<https://mariadb.com/kb/en/built-in-functions/>).

On utilise généralement ces fonctions dans un **SELECT**, et on les reconnaît grâce à l'utilisation de parenthèses.

```
SELECT PI(); -- Fonction qui donne le nombre Pi
```

	123 PI() ▼
1	3.141593

Les arguments

Une fonction peut prendre en entrée un ou plusieurs **arguments**, qui peuvent être de plusieurs types différents.

```
SELECT ROUND(2.45); -- Fonction arrondi
```

123 ROUND(2.45) ▼	
1	2

```
SELECT ROUND(2.45, 1); -- Le deuxième argument  
définit le nombre de décimales après la virgule
```

123 ROUND(2.45, 1) ▼	
1	2.5

Les arguments

Bien entendu, on va fréquemment appliquer ces fonctions sur des **données**, afin de transformer ces dernières (on utilise ici la base de données video_games).

```
SELECT game_platform_id, sale_place, ROUND(quantity, 1)
FROM sales;
```

	123 game_platform_id	A-Z sale_place	123 ROUND(quantity, 1)
1	1	EU	29
2	1	JP	3.8
3	1	NA	41.4
4	1	Other	8.4
5	2	EU	3.6
6	2	JP	6.8
7	2	NA	29.1
8	2	Other	0.8
9	3	EU	12.8
10	3	JP	3.8
11	3	NA	15.7
12	3	Other	3.3
13	4	EU	10.9
14	4	JP	3.3
15	4	NA	15.6

L'**aide sur une fonction** peut s'obtenir avec :
HELP ROUND;

Opérateurs

Les **opérateurs** sont similaires aux fonctions, mais utilisent généralement deux arguments numériques (qui peuvent être des données).

```
SELECT user_count, critic_count, user_count / critic_count  
AS ratio_count  
FROM game_platform;
```

	123 user_count	123 critic_count	123 ratio_count
1	522	51	6.3137
2	[NULL]	[NULL]	[NULL]
3	709	73	9.7123
4	192	73	2.6301
5	[NULL]	[NULL]	[NULL]
6	[NULL]	[NULL]	[NULL]
7	431	65	6.6308
8	129	41	3.1463
9	594	80	7.425
10	[NULL]	[NULL]	[NULL]
11	[NULL]	[NULL]	[NULL]
12	464	64	7.25
13	[NULL]	[NULL]	[NULL]
14	146	63	2.3175
15	106	45	2.3556

Les fonctions peuvent être **combinées** pour donner des résultats riches.

```
SELECT g.name,  
       (gp.critic_score/10) - gp.user_score AS diff_score,  
       GREATEST(gp.critic_score/10, gp.user_score) AS max_score  
FROM game_platform gp JOIN game g ON gp.game_id = g.game_id  
WHERE critic_score IS NOT NULL  
ORDER BY ABS(diff_score) DESC;
```

	A-Z name	123 diff_score	123 max_score
1	Thrillville: Off the Rails	6.5999999762	7.2
2	Shrek Swamp Kart Speedway	-6.19999996185	8.89999996185
3	Company of Heroes 2	5.9000000954	8
4	Call of Duty: Modern Warfare 3	5.5999999523	8.8
5	Call of Duty: Modern Warfare 3	5.3999999046	8.8
6	Call of Duty: Modern Warfare 3	5.3	7.8
7	Thrillville: Off the Rails	5.2999999523	7
8	Call of Duty: Modern Warfare 3	5.2000000477	7
9	Antz Extreme Racing	-5.1999998093	8.69999998093
10	Football Manager Live	5.1000000954	8.2
11	Skylanders: SuperChargers	4.8000000477	8.1
12	Diablo III	4.8	8.8
13	Call of Duty: Ghosts	4.7000000954	7.3
14	Call of Duty: Infinite Warfare	4.7000000954	7.8
15	Shrek Super Party	-4.7	8

SQL possède de nombreuses **fonctions et opérateurs numériques** (pour une liste exhaustive, voir <https://mariadb.com/kb/en/numeric-functions/>) :

- Opérateurs : **+**, **-**, **/**, *****, **%** (modulo)
- Puissance : **POW()**
- Racine carrée : **SQRT()**
- Valeur absolue : **ABS()**
- Arrondi : **ROUND()**
- Exponentielle, logarithme : **EXP()**, **LOG()**

Il existe aussi des **fonctions sur les chaînes de caractères** (voir <https://mariadb.com/kb/en/string-functions/>) :

- Concaténation : **CONCAT()**
- Sous-chaîne : **LEFT()**, **RIGHT()**, **SUBSTR()**
- Retrait des espaces : **TRIM()**
- Mise en majuscules/minuscules : **UPPER()**, **LOWER()**
- Nombre de caractères : **CHAR_LENGTH()**

Et aussi des **fonctions sur les dates**

(<https://mariadb.com/kb/en/date-time-functions/>) :

- La date et/ou l'heure actuelles : **CURDATE()**, **CURTIME()**, **NOW()**
- Ajouter/retirer un intervalle : **DATE_ADD()**, **DATE_SUB()**
- Différence de date : **DATEDIFF()**
- Conversion d'une chaîne de caractères en date :

```
SELECT STR_TO_DATE('10/03/05 14h54', '%d/%m/%y %kh%i') AS d1,  
       STR_TO_DATE('4 31 2012', '%m %d %Y') AS d2;
```

	 d1	 d2
1	2005-03-10 14:54:00.000	2012-05-01

(Pour voir comment "parser" une chaîne : https://mariadb.com/kb/en/str_to_date/)

Les fonctions d'agrégation

Les fonctions d'agrégation

Les **fonctions d'agrégation** sont des fonctions particulières qui effectuent des calculs sur **plusieurs entrées** d'une table.

```
SELECT COUNT(*) FROM game_platform; -- le nombre d'entrées
```

	123 COUNT(*)
1	16,717

```
SELECT COUNT(*) FROM game_platform  
WHERE platform_id = 1; -- le nombre de jeux Wii
```

	123 COUNT(*)
1	1,320

```
SELECT COUNT(DISTINCT game_id) FROM game_platform; -- le nombre  
d'entrées distinctes
```

	123 COUNT(DISTINCT game_id)
1	11,562

Les fonctions d'agrégation numériques

Plusieurs fonctions d'agrégation permettent des calculs sur les **entrées numériques** :

- Le maximum/minimum : **MAX()**, **MIN()**.
- La somme : **SUM()**
- La moyenne : **AVG()**
- La variance et déviation standard : **VARIANCE()**, **STD()**

```
SELECT MAX(critic_score) FROM game_platform;
```

	123 MAX(critic_score) ▼
1	98

Fonctions d'agrégation et WHERE

On peut utiliser les fonction d'agrégation à la suite d'un **WHERE**, pour sélectionner certaines données en fonction d'un calcul.

```
-- Sélection des jeux avec note maximale
SELECT name, critic_score FROM game_platform gp
JOIN game g ON gp.game_id = g.game_id
WHERE critic_score =
(SELECT MAX(critic_score) FROM game_platform);
```

	A-Z name	123 critic_score
1	Grand Theft Auto IV	98
2	Grand Theft Auto IV	98
3	Tony Hawk's Pro Skater 2	98
4	SoulCalibur	98

```
-- Combien de jeux sont en dessous de la moyenne
SELECT COUNT(*) FROM game_platform
WHERE critic_score <
(SELECT AVG(critic_score) FROM game_platform);
```

	123 COUNT(*)
1	3,547

Fonctions d'agrégation et GROUP BY

Avec l'instruction **GROUP BY**, les fonctions d'agrégation permettent des **calculs par groupe**.

```
-- Ventes par région
```

```
SELECT sale_place, SUM(quantity) FROM sales  
GROUP BY sale_place;
```

	A-Z sale_place	123 SUM(quantity)
1	EU	2,424.1399958469
2	JP	1,297.3999979068
3	NA	4,400.8399991505
4	Other	791.2599971443

```
-- Nombre de jeux par plateforme
```

```
SELECT p.name, COUNT(*) FROM game_platform gp  
JOIN platform p ON gp.platform_id = p.platform_id  
GROUP BY gp.platform_id;
```

	A-Z name	123 COUNT(*)
1	Wii	1,320
2	NES	98
3	GB	98
4	DS	2,152

Exemple avec GROUP BY

```
-- Note moyenne par plateforme  
SELECT p.name, AVG(critic_score) AS mean_score FROM game_platform gp  
JOIN game g ON gp.game_id = g.game_id  
JOIN platform p ON gp.platform_id = p.platform_id  
WHERE gp.critic_score IS NOT NULL  
GROUP BY p.name  
ORDER BY mean_score DESC;
```

	A-z name	123 mean_score
1	DC	87.3571
2	PC	75.9287
3	XOne	73.3254
4	PS4	72.0913
5	PS	71.515
6	PSV	70.7917
7	WiiU	70.7333
8	PS3	70.3829
9	XB	69.8593
10	GC	69.4888
11	PS2	68.7273
12	X360	68.6168
13	PSP	67.4242
14	GBA	67.3721
15	3DS	67.1012
16	DS	63.7615
17	Wii	62.8239

Les procédures

Les **procédures** ressemblent aux fonctions dans le sens qu'elles prennent en entrées des arguments et effectuent ensuite une suite d'instructions SQL, mais diffèrent sur plusieurs points :

- Elles sont appelées explicitement avec le mot clé **CALL**.
- Elles peuvent créer de nouveaux objets (p.ex. des tables)
- Elles ne retournent directement aucune valeur.
- Il n'en existe aucune intégrée.

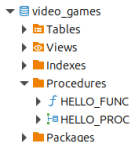
Un appel à une procédure suit généralement la syntaxe suivante :

CALL NOM_DE_PROCEDURE(arguments);

Création de procédures et de fonctions

Avec du code SQL, il est possible de **créer de nouvelles fonctions et procédures** qui seront alors stockées dans la base de données.

Par exemple, en exécutant `proc_func_hello.sql`, on a :



On peut alors les appeler dans un script sur notre base de données.

```
-- Procedure HELLO_PROC, crée une nouvelle table
CALL HELLO_PROC('World par une procédure')
-- Fonction HELLO_FUNC, à utiliser dans une requête
SELECT HELLO_FUNC('World par une fonction');
```

Démo de la procédure UNPIVOT()